

Model

Predictive Failure Model — Complete Roadmap

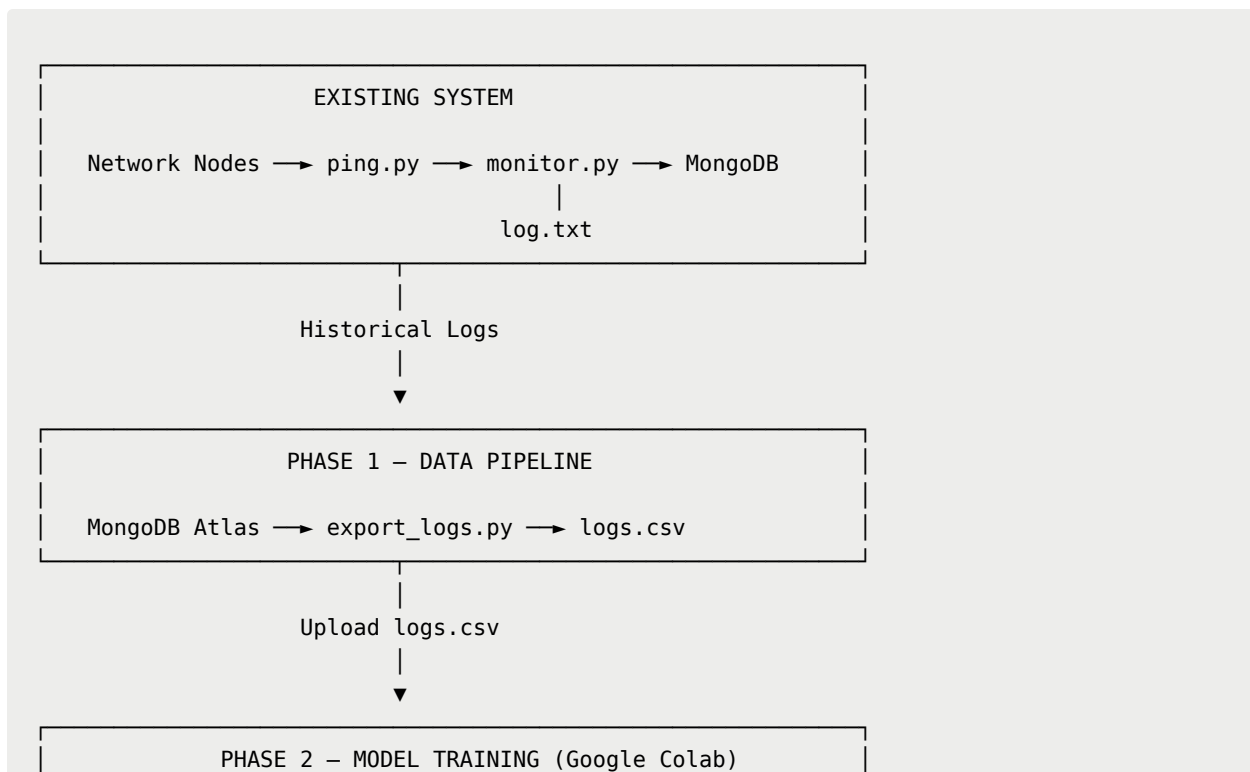
Network Monitoring & Predictive Failure System

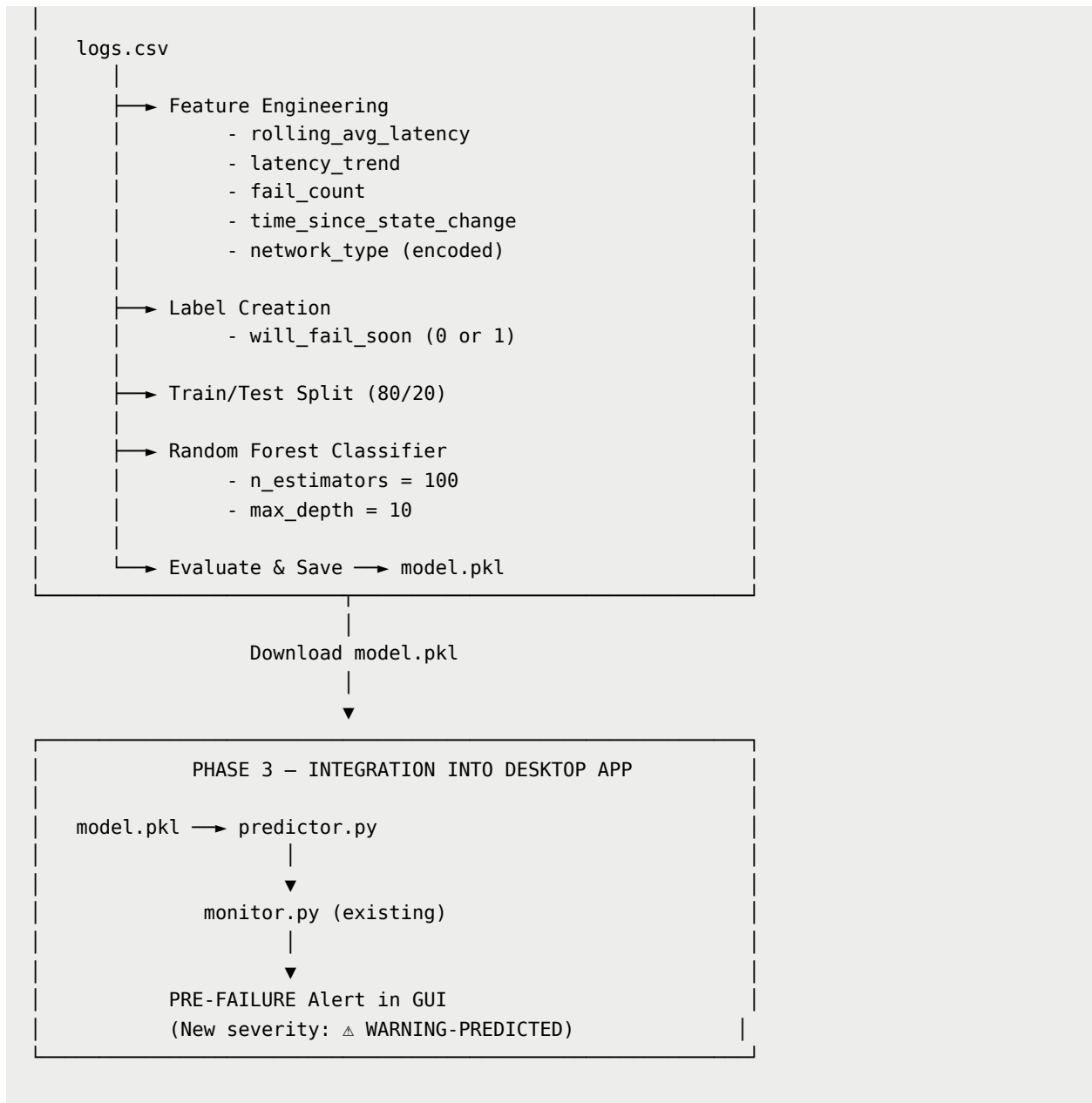
Stack: Python · Random Forest · Google Colab · MongoDB Atlas · PySide6

Project Goal

Predict **network node failure before it happens** using historical log data stored in MongoDB Atlas, trained with a Random Forest classifier in Google Colab, and integrated into the existing desktop monitoring application.

High-Level Architecture Diagram





Detailed Phase Breakdown

PHASE 1 — Data Extraction & Preparation

Goal: Export MongoDB logs into a clean CSV file ready for Colab.

Step 1.1 — Export script (`export_logs.py`)

- Connect to MongoDB Atlas using existing `MONGO_URI` from `.env`

- Query the `logs` collection (all documents)
- Convert to Pandas DataFrame
- Save as `logs.csv`

Fields exported:

Field	Type	Description
timestamp	datetime	When the log was recorded
node_name	string	Name of the network node
ip	string	IP address of node
network_type	string	wifi / router
state	string	UP / DEGRADED / DOWN
latency	float	Ping latency in ms
fails	int	Consecutive failure count

Step 1.2 — Data Cleaning (in Colab)

- Remove rows with null latency
- Sort by `node_name` + `timestamp`
- Encode `network_type` as integer (wifi=0, router=1)
- Encode `state` as integer (UP=0, DEGRADED=1, DOWN=2)

🟡 PHASE 2 — Feature Engineering

Goal: Create meaningful features that help the model detect patterns BEFORE failure.

Step 2.1 — Rolling Features (per node)

Feature	Formula	Why It Matters
<code>rolling_avg_latency_5</code>	Mean of last 5 latency readings	Rising average = degradation trend
<code>rolling_max_latency_5</code>	Max of last 5 readings	Spike detection
<code>latency_trend</code>	Current - Previous latency	+ve = getting worse

Feature	Formula	Why It Matters
<code>fail_count</code>	Consecutive failures	Direct failure indicator
<code>state_encoded</code>	UP=0, DEGRADED=1, DOWN=2	Current node health
<code>network_type_encoded</code>	wifi=0, router=1	Node type matters
<code>time_in_current_state</code>	Seconds since last state change	Long DEGRADED = likely DOWN soon

Step 2.2 — Label Creation (Target Variable)

```
will_fail_soon = 1  if the node goes DOWN within the next 3 log entries
will_fail_soon = 0  otherwise
```

This is the value we want the model to **predict**.

● PHASE 3 — Model Training (Google Colab)

Goal: Train a Random Forest Classifier to predict `will_fail_soon`.

Step 3.1 — Train/Test Split

```
80% Training Data → Model learns patterns
20% Testing Data  → We evaluate accuracy
```

Step 3.2 — Random Forest Parameters

Parameter	Value	Meaning
<code>n_estimators</code>	100	100 decision trees vote together
<code>max_depth</code>	10	Each tree can go 10 levels deep
<code>random_state</code>	42	Reproducible results
<code>class_weight</code>	balanced	Handles rare DOWN events fairly

Step 3.3 — Evaluation Metrics

Metric	Target	Why
Accuracy	> 85%	Overall correctness
Precision	> 80%	Don't false alarm too much
Recall	> 85%	Don't miss real failures
F1 Score	> 82%	Balance of precision + recall
Confusion Matrix	Visualized	See exactly what it gets wrong

Step 3.4 — Feature Importance Plot

- Bar chart showing which features matter most
- Expected: `fail_count` and `rolling_avg_latency_5` will rank highest

Step 3.5 — Save Model

```
import pickle
with open("model.pkl", "wb") as f:
    pickle.dump(model, f)
```

● PHASE 4 — Integration into Desktop App

Goal: Load `model.pkl` into the existing system and generate PRE-FAILURE alerts.

Step 4.1 — New File: `predictor.py`

- Loads `model.pkl` on startup
- Accepts current node features as input
- Returns prediction: `0 = Safe` or `1 = Failure Predicted`

Step 4.2 — Update `monitor.py`

- After each ping, pass features to `predictor.py`
- If prediction = 1, generate a new alert type

Step 4.3 — New Alert Severity: `PRE-FAILURE`

State	Severity	Color
UP (Recovery)	INFO	 Green
DEGRADED	WARNING	 Yellow
FAILURE PREDICTED	PRE-FAILURE	 Orange
DOWN	CRITICAL	 Red

Step 4.4 — GUI Update (`alerts.py`)

- Add orange color for `PRE-FAILURE` alerts
- Display message: `"⚠ [PRE-FAILURE] node_1_phone – Failure predicted in next cycle"`

New Files to Create

```

project/
├── src/
│   ├── predictor.py      ← NEW: loads model, runs prediction
│   ├── export_logs.py    ← NEW: exports MongoDB → CSV
│   └── monitor.py        ← MODIFIED: calls predictor
├── model/
│   └── model.pkl          ← NEW: trained Random Forest model
└── colab/
    └── train_model.ipynb  ← NEW: Google Colab notebook

```

Google Colab Notebook Structure

```

train_model.ipynb
├── Cell 1: Install libraries (pandas, sklearn, matplotlib, seaborn)
├── Cell 2: Upload & load logs.csv
├── Cell 3: Data cleaning
├── Cell 4: Feature engineering
├── Cell 5: Label creation (will_fail_soon)
├── Cell 6: Train/Test split
├── Cell 7: Train Random Forest
├── Cell 8: Evaluate (accuracy, F1, confusion matrix)
├── Cell 9: Feature importance plot
└── Cell 10: Save & download model.pkl

```

Complete Data Flow Diagram





Suggested Timeline

Phase	Task	Estimated Time
Phase 1	Export script + CSV generation	1 day
Phase 2	Feature engineering in Colab	1 day
Phase 3	Model training + evaluation	1–2 days
Phase 4	Integration into desktop app	2 days
Testing	End-to-end testing	1 day
Total		~6–7 days



Libraries Required

Local (already have):

- `pymongo` , `python-dotenv` , `pandas`

Google Colab (install in notebook):

```
pip install pandas scikit-learn matplotlib seaborn imbalanced-learn pickle5
```

New addition to project:

```
pip install scikit-learn
```



Success Criteria

- ☐ Model accuracy > 85%
- ☐ PRE-FAILURE alert fires at least 1 cycle before node goes DOWN
- ☐ No false alarms more than 10% of the time
- ☐ Model loads in < 1 second on desktop app startup
- ☐ Works offline (no internet needed after model is trained)



Future Improvements (Post MVP)

- Retrain model automatically every week with new logs
- Add more nodes to improve training data diversity
- Try LSTM for even better time-series prediction
- Add prediction confidence score (%) to the alert message
- Web dashboard showing predicted failure probability per node

Document prepared for mentor presentation / Network Monitoring & Predictive Failure System